

**Universidad Nacional de Ingeniería
Facultad de Ciencias**

Arquitectura de computadores

Instrucciones del computador basado en MIPS

Prof.: Lic. César Martín Cruz S.
ccruz@uni.edu.pe

Aritmética MIPS

- Todas las instrucciones aritméticas tienen 3 operandos.
- El orden de los operandos es fija (primero el operando destino)

Ejemplo:

C: $A = B + C$

MIPS: `add $s0, $s1, $s2` $\# \$s0 = \$s1 + \$s2$

Aritmética MIPS

- La simplicidad favorece la regularidad.

C: $A = B + C + D;$

$E = F - A;$

MIPS: `add $t0, $s1, $s2` $\# \$t0 = \$s1 + \$s2$
 `add $s0, $t0, $s3` $\# \$s0 = \$t0 + \$s3$
 `sub $s4, $s5, $s0` $\# \$s4 = \$s5 - \$s0$

- Los operandos de las instrucciones aritméticas tienen que ser registros.

Registros

- Un registro es una memoria integrada en la CPU.
- Su capacidad es de 4 bytes (32 bits) y de acceso muy rápido.
- Se usan para guardar valores temporales y resultados de operaciones aritméticas.
- Datos de 32 bits son llamados “word” o “palabra”.
- Los nombres del programa ensamblador para valores temporales son: \$t0, \$t1,, \$t9.
- Para variables salvadas o guardadas \$s0, \$s1, ..., \$s7.

Registros vs Memoria

- Solo hay 32 registros disponibles en MIPS.
- El compilador asocia las variables con los registros.
- Las variables se guardan en la memoria principal.
- La mayoría de las CPUs modernas mueven las variables de la memoria principal a los registros, operan sobre ellos y regresan el resultado a la memoria.
- A esto se le conoce como **arquitectura load/store**.
- MIPS emplea la arquitectura **load/store**.

Organización de la memoria

- MIPS usa *byte addressing*, el índice apunta a un byte de memoria.

0	8 bits of data
1	8 bits of data
2	8 bits of data
3	8 bits of data
4	8 bits of data
5	8 bits of data

...

Organización de la memoria

- Las palabras son de 4 bytes (32 bits).
- Los registros son de 4 bytes (32 bits).
- La memoria está **alineada**. Las palabras deben comenzar en direcciones que son múltiplo de 4.

0	Datos 32 bits
4	Datos 32 bits
8	Datos 32 bits
12	Datos 32 bits
...	

Organización de la memoria

- MIPS se puede configurar por hardware como big-endian(byte más significativo en la dirección baja de la palabra) o little-endian.
- Se usan 32 bits para direccionar la memoria.
- La memoria tiene:
 - 2^{32} bytes con direcciones desde 0 hasta $2^{32} - 1$.
 - 2^{30} palabras con direcciones desde 0 hasta $2^{32} - 4$.

Instrucciones de referencia a memoria

- Instrucciones lw (load word) y sw (store word).

C: $A[8] = h + A[8];$

MIPS:	lw \$t0, 32(\$s3)	; t0 = Memoria[s3 + 32]
	add \$t0, \$s2, \$t0	; t0 = s2 + t0
	sw \$t0, 32(\$s3)	; Memoria[s3 + 32] = t0

s2 valor de h
s3 dirección A[0]
t0 valor de A[8]

Ojo:

lw – el destino va **primero**.
sw – el destino va **segundo**.

Operandos de memoria

- En las instrucciones de memoria, a la constante se le llama *offset* y al registro que se suma para obtener la dirección se le llama *registro base*.
- Por ejemplo:
 lw \$t0, 32(\$s3) ; \$t0 = Memoria[\$s3 + 32]
 sw \$t0, 32(\$s3) ; Memoria[\$s3 + 32] = \$t0
- 32 es el offset.
- El registro base es \$s3.

En Resumen

- MIPS

- Carga palabras pero direcciona bytes.
- Puede ser big-endian o little-endian.
- Aritmética usa solamente registros.

<u>Instrucción</u>	<u>Significado</u>
add \$s1, \$s2, \$s3	$\$s1 = \$s2 + \$s3$
sub \$s1, \$s2, \$s3	$\$s1 = \$s2 - \$s3$
lw \$s1, 100(\$s2)	$\$s1 = \text{Memoria}[\$s2+100]$
sw \$s1, 100(\$s2)	$\text{Memoria}[\$s2+100] = \$s1$

Ejemplo 1:

```
swap(int v[], int k)
{ int temp;
  temp = v[k];
  v[k] = v[k+1];
  v[k+1] = temp;
}
```

swap:

```
mulh $2, $5, 4
add $2, $4, $2
lw $15, 0($2)
lw $16, 4($2)
sw $16, 0($2)
sw $15, 4($2)
jr $31
```

Lenguaje de Máquina

- El lenguaje de máquina es lenguaje binario.
- El hardware solo entiende bits.
- El ensamblador traduce cada instrucción de lenguaje ensamblador a lenguaje de máquina.
- Por ejemplo, la instrucción
add \$t0, \$s1, \$s2
se traduce a 00000010001100100100100000100000

Lenguaje de Máquina

00000010001100100100100000100000 se descompone en:

- 000000 es el opcode (código de operación) del add.
- 10001 es el número del registro \$s1.
- 10010 es el número del registro \$s2.
- 01001 es el número del registro \$t0.
- 00000 son bits de relleno (don't care).
- 100000 es el código de la suma de registros.

Lenguaje de Máquina

En general, la instrucción

add destino, fuente1, fuente2

genera el código máquina

000000ffffgggggdddddxxxxx100000

Donde:

1. 000000 es el código del add.
2. ffff es el número del registro fuente 1
3. ggggg es el número del registro fuente 2
4. dddd es el número del registro destino
5. xxxxx son para no tener cuidado (don't care)
6. 100000 es el código de la suma de registros.

Formato de Instrucción

Hay 3 formatos de instrucción:

- **Formato R.** Todos los operandos son registros.
- **Formato I.** Hay un operando inmediato (número).
- **Formato J.** La instrucción es un salto (jump).

Ejemplos:

- Formato R.

add \$t0, \$s1, \$s2 ; $\$t0 = \$s1 + \$s2$

- Formato I.

addi \$t0, \$s1, C ; $\$t0 = \$s1 + C$ [número de 16 bits]

- Formato J.

j C ; salta a C [dirección de 16 bits]

Instrucciones de control

- Instrucciones de toma de decisión.
- Alteran el flujo de control.
- En otras palabras, cambian la siguiente instrucción que será ejecutada.

En MIPS hay dos instrucciones de salto condicional:

1. `beq $s, $t, C` ; salta a la dirección C si $\$s == \t .

2. `bne $s, $t, C` : salta a la dirección C si $\$s \neq \t .

Ejemplo:

C: `if (i == j)`
 `h = i + j;`

MIPS: `bne $t1, $t2, L1` ; \$t1: valor de i, \$t2: valor de j
 `add $t0, $t1, $t2` ; \$t0: valor de h

L1:...

Instrucciones de control

Ejemplo:

```
C:      if (i != j)
        h = i + j;
      else
        h = i - j;
```

```
MIPS: bne $s1, $s2, L1 ; $s1 = i, $s2 = j
      sub $s0, $s1, $s2 ; $s0 = h
      j L2 ; salta al final
L1: add $s0, $s1, $s2 ; $s0 = h
L2: ...
```

Instrucciones de control

En MIPS hay tres instrucciones de salto incondicional:

1.j C ; salta a la dirección C

2.jr \$r ; salta a la dirección guardada en \$r

3.jal C ; llama al procedimiento que comienza en C
; la dirección de regreso se guarda en \$31
; es decir, se regresa con jr \$31

Instrucciones de control

Ejemplo:

```
C    foo ();  
    ...  
    void foo ()  
    {  
    ...  
    }
```

```
MIPS:    jal foo    ; salta a foo, guarda la dirección de  
          ...        ; regreso en el registro 31  
foo:     ...  
          jr $31     ; regresa al programa principal
```

Instrucciones de control

- MIPS tiene beq (salta si es igual) y bne (salta si no es igual).
¿Y si la condición es **if (i < j)**?
- Existe la instrucción slt (set if less than):
slt \$t0, \$t1, \$t2
- slt es equivalente en pseudo C a
 $\$t0 = (\$t1 < \$t2)$
- \$t0 vale 1 si \$t1 es menor que \$t2 y vale 0 en otro caso.
- slt se usa junto con beq o bne para condiciones **if (i < j)**.

Instrucciones de control

Ejemplo:

C: if (i < j)
 h = i + j;
 else
 h = i - j;

MIPS: slt \$s3, \$s1, \$s2 ; \$s1 = i, \$s2 = j \$s3 = i < j
 beq \$s3, \$zero, L1 ; \$zero siempre vale 0
 add \$s0, \$s1, \$s2 ; entonces h = i + j
 j L2 ; salta al final (L2)
 L1: sub \$s0, \$s1, \$s2 ; sino h = i - j
 L2:...

Instrucciones de control

Ejemplo:

C: if (i <= j)
 h = i + j;
 else
 h = i - j;

MIPS:	beq \$s1, \$s2, L3	; salta si \$s1 == \$s2
	slt \$s3, \$s1, \$s2	; chequea si \$s1 < \$s2
	beq \$s3, \$zero, L1	; \$zero siempre vale 0
L3:	add \$s0, \$s1, \$s2	; entonces \$s0 = \$s1 + \$s2
	j L2	; salta al final (L2)
L1:	sub \$s0, \$s1, \$s2	; sino \$s0 = \$s1 - \$s2
L2:...		

Resumen

Instrucción

add \$s1,\$s2,\$s3

sub \$s1,\$s2,\$s3

lw \$s1,100(\$s2)

sw \$s1,100(\$s2)

bne \$s4,\$s5,L

beq \$s4,\$s5,L

j L

jal L

Significado

$\$s1 = \$s2 + \$s3$

$\$s1 = \$s2 - \$s3$

$\$s1 = \text{Memoria}[\$s2+100]$

$\text{Memoria}[\$s2+100] = \$s1$

Siguiente instrucción es L si $\$s4 \neq \$s5$

Siguiente instr. es L si $\$s4 = \$s5$

Siguiente instr. es L

Llama al procedimiento L

Constantes

- Las instrucciones con formato I (inmediato), tienen una constante en su tercer operando.
- Las constantes están limitadas a 16 bits **con signo**.

- Por ejemplo:

addi \$29, \$29, 4	; $\$29 = \$29 + 4$
slti \$8, \$18, 10	; $\$8 = \$18 < 10$
andi \$29, \$29, 6	; $\$29 = \$29 \& 6$
ori \$29, \$29, 0xFF	; $\$29 = \$29 0xFF$

Constantes

- Se pueden tener constantes de 32 bits, pero solo si se almacenan en un registro.
- Se necesitan dos instrucciones (o una pseudo instrucción, como se verá después):
 1. `lui $x, C` ; guarda C en los 16 bits mas significativos del registro \$x
 2. `ori $x, C` ; guarda C en los 16 bits menos significativos del registro \$x
- En lugar de `ori` se puede usar:
`addi $x, $zero, C`

Operaciones lógicas

- Instrucciones para manipulación de bits.

Operación	C	Java	MIPS
Shift left	<<	<<	sll
Shift right	>>	>>>	srl
Bitwise AND	&	&	and, andi
Bitwise OR			or, ori
Bitwise NOT	~	~	nor

- Útiles para la extracción y la inserción de grupos de bits en una palabra

Operación Shift (Desplazamiento)

op	rs	rt	rd	shamt	funct
6 bits	5 bits	5 bits	5 bits	5 bits	6 bits

- Shamt: Cuantas posiciones a desplazar
- Shift left logical
 - Desplazamiento a la izquierda y se llena con bits 0s
 - sll by i bits multiplica por 2^i
- Shift right logical
 - Desplazamiento a la derecha y se llena con bits 0s
 - srl by i bits divide por 2^i (solamente números sin signo)

Operación AND

- Útil para enmascarar los bits en una palabra
 - Selecciona algunos bits, los pone 0s a otros

and \$t0, \$t1, \$t2

\$t2	0000 0000 0000 0000 0000 1101 1100 0000
\$t1	0000 0000 0000 0000 0011 1100 0000 0000
\$t0	0000 0000 0000 0000 0000 1100 0000 0000

Operación OR

- Útil para incluir bits en una palabra
 - Se fijan algunos bits a 1, deja a otros intercambiables

or \$t0, \$t1, \$t2

\$t2	0000 0000 0000 0000 0000 1101 1100 0000
\$t1	0000 0000 0000 0000 0011 1100 0000 0000
\$t0	0000 0000 0000 0000 0011 1101 1100 0000

Operación NOT

- Útil para invertir los bits en una palabra
 - Cambia 0 a 1, y 1 a 0
- MIPS tiene una instrucción de 3 operandos NOR
 - $a \text{ NOR } b == \text{NOT} (a \text{ OR } b)$

`nor $t0, $t1, $zero`

Registro 0: siempre se refiere a un cero

\$t1 0000 0000 0000 0000 0011 1100 0000 0000

\$t0 1111 1111 1111 1111 1100 0011 1111 1111

Haciendo un loop en MIPS

■ En C:

```
while (save[i] == k) i += 1;
```

■ i es \$s3, k es \$s5, dirección de save en \$s6

■ En MIPS:

```
Loop: sll  $t1, $s3, 2  
      add  $t1, $t1, $s6  
      lw   $t0, 0($t1)  
      bne  $t0, $s5, Exit  
      addi $s3, $s3, 1  
      j    Loop
```

```
Exit: ...
```

Con signo y sin signo

- Comparación con signo: slt, slti
- Comparación sin signo: sltu, sltui

- Ejemplo:

- \$s0 = 1111 1111 1111 1111 1111 1111 1111 1111

- \$s1 = 0000 0000 0000 0000 0000 0000 0000 0001

- slt \$t0, \$s0, \$s1 # con signo

- $-1 < +1 \Rightarrow \$t0 = 1$

- sltu \$t0, \$s0, \$s1 # sin signo

- $+4,294,967,295 > +1 \Rightarrow \$t0 = 0$

Uso de Registros

- \$a0 – \$a3: argumentos (registros de 4 – 7)
- \$v0, \$v1: valores de resultado (registros 2 y 3)
- \$t0 – \$t9: temporales
 - Pueden ser sobreescritos por el destinatario
- \$s0 – \$s7: guardados
 - Deben ser guardados/restaurados por el destinatario
- \$gp: puntero global para datos estáticos (registro 28)
- \$sp: puntero de pila (registro 29)
- \$fp: puntero de marco (registro 30)
- \$ra: dirección de retorno (registro 31)

Los datos : Caracteres

- Conjunto de caracteres codificados como byte
 - ASCII: 128 caracteres
 - 95 gráficos, 33 control
 - Latin-1: 256 caracteres
 - ASCII, +96 mas caracteres gráficos
- Unicode: conjunto de caracteres de 32-bits
 - Usado en Java, C++, ...
 - Cubre la mayoría de los alfabetos del mundo, además de símbolos
 - UTF-8, UTF-16: codificaciones de longitud variable

Operaciones byte/media palabra

- Podría utilizar operaciones bit a bit
- MIPS: load/store de byte/halfword
 - Procesamiento de una cadena es un caso común
lb rt, offset(rs) lh rt, offset(rs)
 - Signo extendido a 32 bits en rt
lbu rt, offset(rs) lhu rt, offset(rs)
 - Cero extendido a 32 bits en rt
sb rt, offset(rs) sh rt, offset(rs)
 - Almacena solo a la derecha el byte/halfword

Ejemplo de copia de una cadena

- Código C :
 - Cadena terminada en Null

```
void strcpy (char x[], char y[])  
{ int i;  
  i = 0;  
  while ((x[i]=y[i])!='\0')  
    i += 1;  
}
```

- Direcciones de x, y en \$a0, \$a1
- i en \$s0